

Procesador RISC-V con Arquitectura Pipelined (Segmentada)

Informe Técnico Completo

Materia: Arquitectura de Computadoras

Tema: Implementación de un Procesador RISC-V RV32I
con Pipeline de 5 Etapas, Forwarding y Hazard Detection

16 de junio de 2026

Índice

1. Introducción

1.1. ¿Qué es un Procesador Pipelined?

Un procesador **pipelined** (segmentado) divide la ejecución de cada instrucción en varias etapas independientes que trabajan en paralelo. Mientras una instrucción está siendo decodificada, la siguiente ya está siendo leída de la memoria, y la anterior ya está realizando su operación aritmética.

Teoría del Paralelismo a Nivel de Instrucción (ILP): El pipelining es la forma más pura de ILP. En un procesador **de ciclo único**, cada instrucción tarda un ciclo completo (cuyo reloj se calibra según la instrucción más lenta, como **lw**). La latencia total y el periodo del reloj limitan drásticamente la frecuencia (MHz/GHz). En contraste, el pipeline divide ese largo camino combinacional en **5 etapas** más cortas separadas por registros.

Esto reduce drásticamente el periodo del reloj y aumenta la frecuencia. Idealmente, el **throughput** (rendimiento) se incrementa por un factor igual al número de etapas ($k = 5$). Aunque la *latencia* de una instrucción individual no mejora (incluso empeora ligeramente por el retardo de los registros), el hecho de que termine **una instrucción por cada ciclo de reloj** hace que el procesador sea inmensamente más rápido a gran escala.

1.2. Teoría de los Riesgos (Hazards)

Al solapar instrucciones, se rompe la ilusión de ejecución secuencial pura, creando **riesgos** que pueden corromper el estado del procesador si no se manejan:

- **Riesgos Estructurales (Structural Hazards):** Ocurren cuando dos etapas distintas intentan acceder al mismo recurso de hardware simultáneamente. En nuestra arquitectura, esto se resuelve con la *Arquitectura Harvard*, teniendo memorias separadas para Instrucciones (Fetch) y Datos (Memory), evitando colisiones.
- **Riesgos de Datos (Data Hazards):** Surgen cuando una instrucción depende del resultado de una instrucción anterior que aún no ha terminado de cruzar el pipeline. Específicamente la dependencia RAW (Read After Write).
- **Riesgos de Control (Control Hazards):** Causados por instrucciones de salto (**beq**, **jal**). Al estar en Fetch, el procesador no sabe si se debe saltar o no hasta la etapa de Execute. Si se equivoca, ejecutará instrucciones incorrectas que deben ser anuladas.

1.3. Las 5 Etapas del Pipeline

1. **Fetch (IF):** Lectura de la instrucción desde la memoria de instrucciones.
2. **Decode (ID):** Decodificación de la instrucción, lectura de registros y generación de señales de control.
3. **Execute (EX):** Ejecución de la operación aritmético-lógica en la ALU.

4. **Memory (MEM)**: Acceso a la memoria de datos.
5. **Writeback (WB)**: Escritura del resultado final en el banco de registros.

1.4. Diagrama Temporal del Pipeline

El siguiente diagrama muestra cómo 5 instrucciones se solapan en el pipeline:

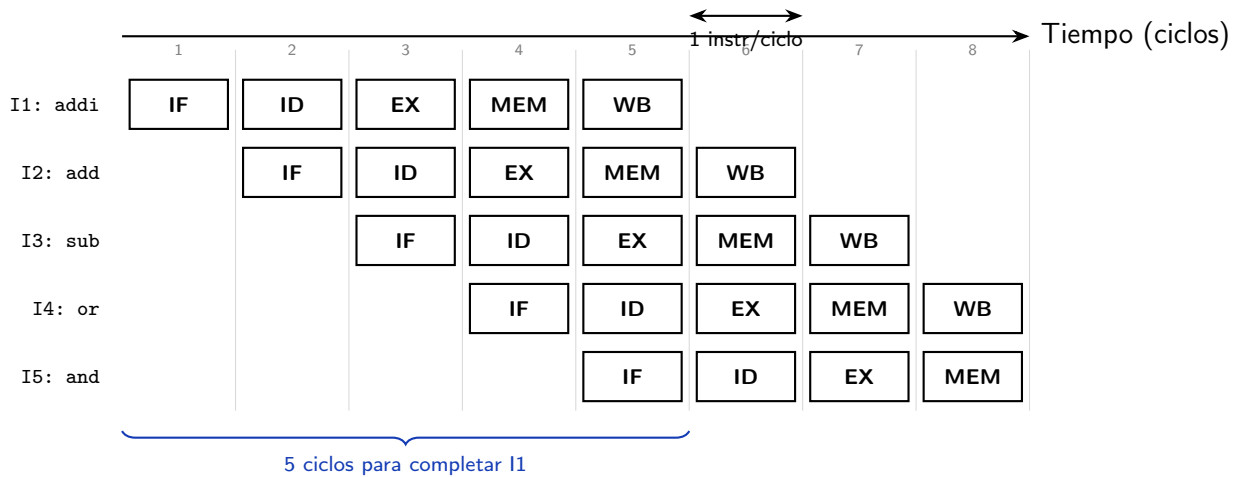


Figura 1: Diagrama temporal: cada instrucción tarda 5 ciclos, pero se completa 1 instrucción por ciclo.

1.5. ISA Implementada: RISC-V RV32I

Tipo	Instrucciones	Opcode	Ejemplo
R-type	add, sub, and, or, sll, slt	0110011	add x4, x2, x3
I-type	addi, lw, ori, slti	0010011 / 0000011	addi x2, x0, 5
S-type	sw	0100011	sw x7, 100(x3)
B-type	beq	1100011	beq x4, x0, L
J-type	jal	1101111	jal x3, func

Cuadro 1: Instrucciones soportadas por el procesador.

2. Los Registros Pipeline

2.1. ¿Qué son y por qué son necesarios?

Los registros pipeline **separan físicamente** una etapa de la siguiente. Sin ellos, las señales se mezclarían. Se ubican entre cada par de etapas.

2.2. Tres Tipos de Flip-Flops

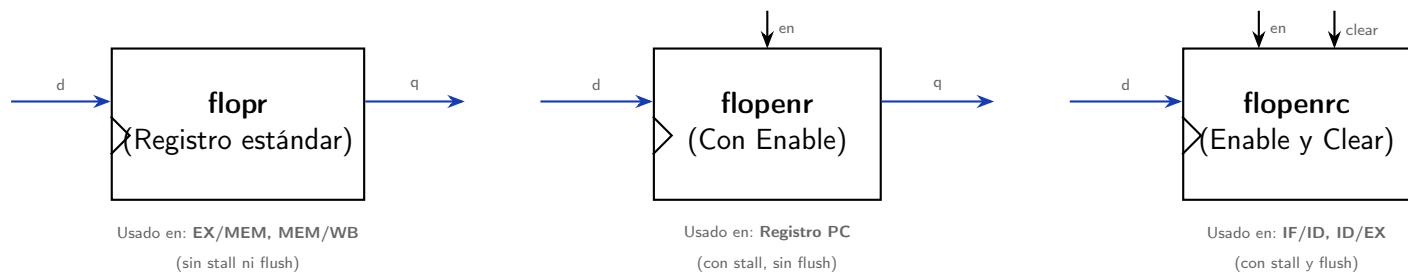


Figura 2: Los tres tipos de flip-flops usados. Observa el triángulo símbolo de la señal de reloj sincronizada por flanco.

2.3. ¿Por qué IF/ID usa flopenrc y MEM/WB usa flopr?

Las etapas tempranas (Fetch, Decode, Execute) son **especulativas**: las instrucciones pueden ser descartadas (*flush*) si un salto cambia el flujo, o detenidas (*stall*) si hay una dependencia de datos sin resolver.

Las etapas tardías (Memory, Writeback) ejecutan instrucciones que ya **pasaron todos los filtros**: ya se resolvieron los saltos y los riesgos. Nunca hay razón para detenerlas ni vaciarlas, de ahí que solo requieran un *flopr* básico.

```

1 // flopenrc: Tiene clk, reset, enable y clear
2 en cada flanco de subida de clk:
3   si reset:      q = 0
4   sino si clear: q = 0 // FLUSH: inserta burbuja NOP
5   sino si en:    q = d // Avanza normalmente
6   // si en = 0:  q no cambia (CONGELADO - STALL)

```

Listing 1: Comportamiento de los registros pipeline

3. Módulo Top-Level: riscv_pipeline

El módulo *riscv_pipeline* es el **ensamblaje principal** que conecta las 5 etapas con la Hazard Unit. Es puramente estructural, cableando todos los submódulos.

```

1 modulo riscv_pipeline(clk, reset):
2   // 1. Instanciar Fetch
3   fetchStage(clk, reset, StallF, PCSrcE, PCTargetE ...)
4
5   // 2. Instanciar Decode
6   decodeStage(clk, reset, StallD, FlushD, ...)
7
8   // 3. Instanciar Execute
9   executeStage(clk, reset, FlushE, ForwardAE, ForwardBE, ...)
10
11  // 4. Instanciar Memory

```

```

12 memStage(clk, reset, senales_Execute ...)
13
14 // 5. Instanciar Writeback
15 wbStage(clk, reset, senales_Memory ...)
16
17 // 6. Instanciar Hazard Unit
18 hazardUnit(Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW, ...)

```

Listing 2: Pseudocódigo del módulo top-level

4. Etapa 1: Fetch (IF)

4.1. Componentes Activos

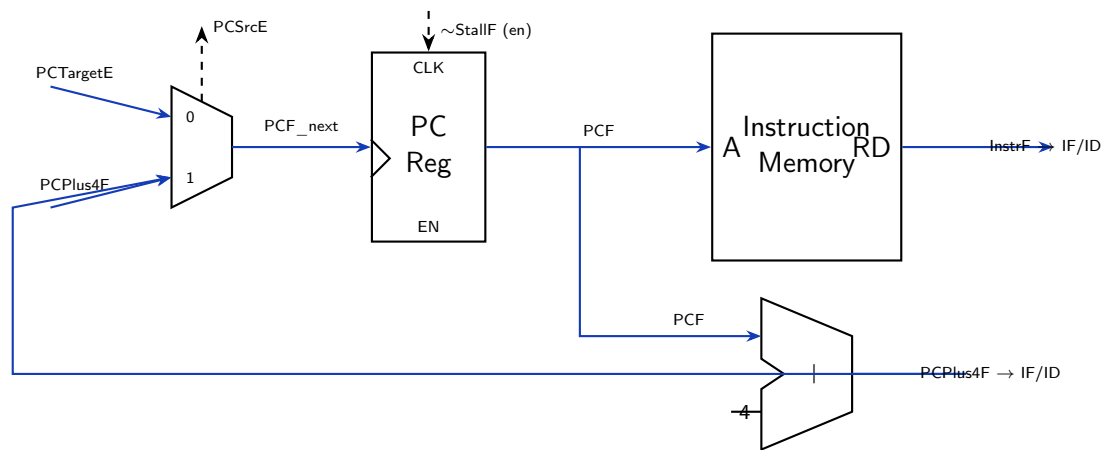


Figura 3: Etapa Fetch aislada con sus componentes operativos.

4.2. Pseudocódigo

```

1 FETCH(clk, reset, StallF, PCSrcE, PCTargetE):
2   // 1. Seleccionar el proximo PC
3   si PCSrcE == 1:
4     PCF_next = PCTargetE      // Salto tomado
5   sino:
6     PCF_next = PCPlus4F      // Secuencial (PC + 4)
7
8   // 2. Actualizar el PC (solo si no hay stall)
9   en cada flanco de subida de clk:
10    si reset:                PCF = 0
11    sino si !StallF:        PCF = PCF_next
12
13   // 3. Leer la instruccion de la memoria
14   InstrF = InstrMemoria[PCF / 4]
15
16   // 4. Calcular PC + 4
17   PCPlus4F = PCF + 4

```

Listing 3: Pseudocódigo de la etapa Fetch

5. Etapa 2: Decode (ID)

5.1. Componentes Activos

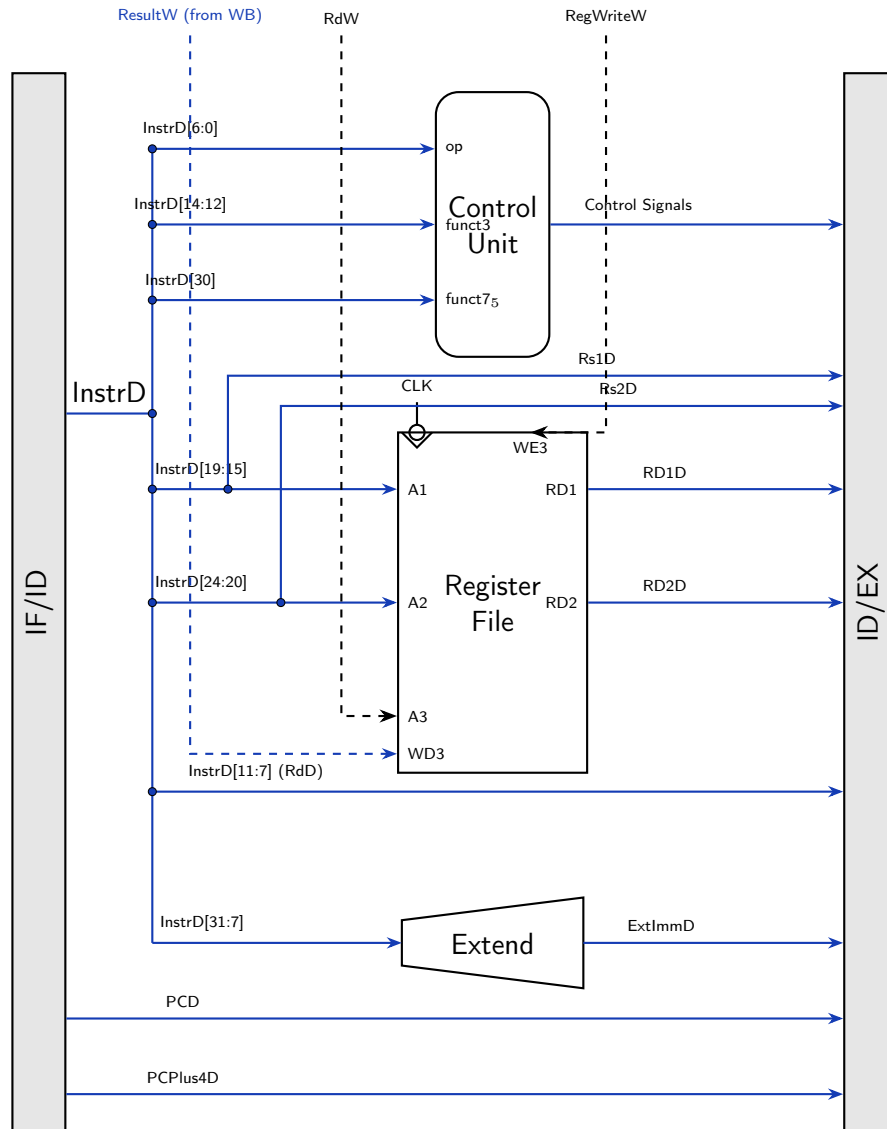


Figura 4: Etapa Decode aislada. Incluye el banco de registros y unidad de control.

5.2. Unidad de Control

La Unidad de Control recibe el opcode, funct3 y funct7 de la instrucción y genera todas las señales de control.

Tipo	Opcode	RegW	ImmSrc	ALUSrc	MemW	ResSrc	Branch	ALUOp
lw	0000011	1	00	1	0	01	0	00
sw	0100011	0	01	1	1	–	0	00
R-type	0110011	1	xx	0	0	00	0	10
I-type	0010011	1	00	1	0	00	0	10
beq	1100011	0	10	0	0	–	1	01
jal	1101111	1	11	0	0	10	0	xx

Cuadro 2: Tabla de verdad de la unidad de control principal.

Nota crítica sobre el ALUControl: El ALUControl usa **4 bits**, no 3. El bit [3] diferencia ADD (0000) de SUB (1000). Si se truncara a 3 bits, la resta se interpretaría como suma y fallaría la operación sub o la lógica de comparación de beq.

5.3. Pseudocódigo de Decode

```

1 DECODE(clk, reset, StallD, FlushD, InstrF, PCF, PCPlus4F):
2   // 1. Registros IF/ID (flopenrc: con stall y flush)
3   en cada flanco de subida de clk:
4     si reset o FlushD:
5       InstrD=0; PCD=0; PCPlus4D=0    // Insertar NOP
6     sino si !StallD:
7       InstrD=InstrF; PCD=PCF; PCPlus4D=PCPlus4F
8
9   // 2. Extraer campos
10  Rs1D = InstrD[19:15]; Rs2D = InstrD[24:20]; RdD = InstrD[11:7]
11
12  // 3. Leer banco de registros (combinacional)
13  RD1D = (Rs1D != 0) ? Registros[Rs1D] : 0
14  RD2D = (Rs2D != 0) ? Registros[Rs2D] : 0
15
16  // 4. Generar senales de control
17  {RegWriteD, MemWriteD, ALUSrcD, ...} = Decodificar(InstrD)
18
19  // 5. Extender inmediato
20  ExtImmD = ExtenderSigno(InstrD[31:7], ImmSrcD)

```

Listing 4: Pseudocódigo de la etapa Decode

6. Etapa 3: Execute (EX)

La etapa Execute es el núcleo de cálculo del procesador. Es aquí donde la ALU (Arithmetic Logic Unit) procesa los datos y toma decisiones críticas para el flujo del programa.

Teoría Operativa: A diferencia de un procesador de ciclo único donde la decisión de saltar (Branch) se evalúa en Decode, en este pipeline se evalúa en Execute. La ALU realiza una resta (sub) entre los dos operandos y levanta la bandera ZeroE si son iguales. Esta bandera, al operarse mediante una compuerta AND con la señal BranchE, determina

la señal PCSrcE (Branch Taken). Este retraso arquitectónico significa que el salto no se resuelve hasta el tercer ciclo, lo que introduce una penalización de 2 ciclos (burbujas) en caso de que el salto sea tomado, los cuales son gestionados puramente por la *Hazard Unit*.

6.1. Componentes Activos

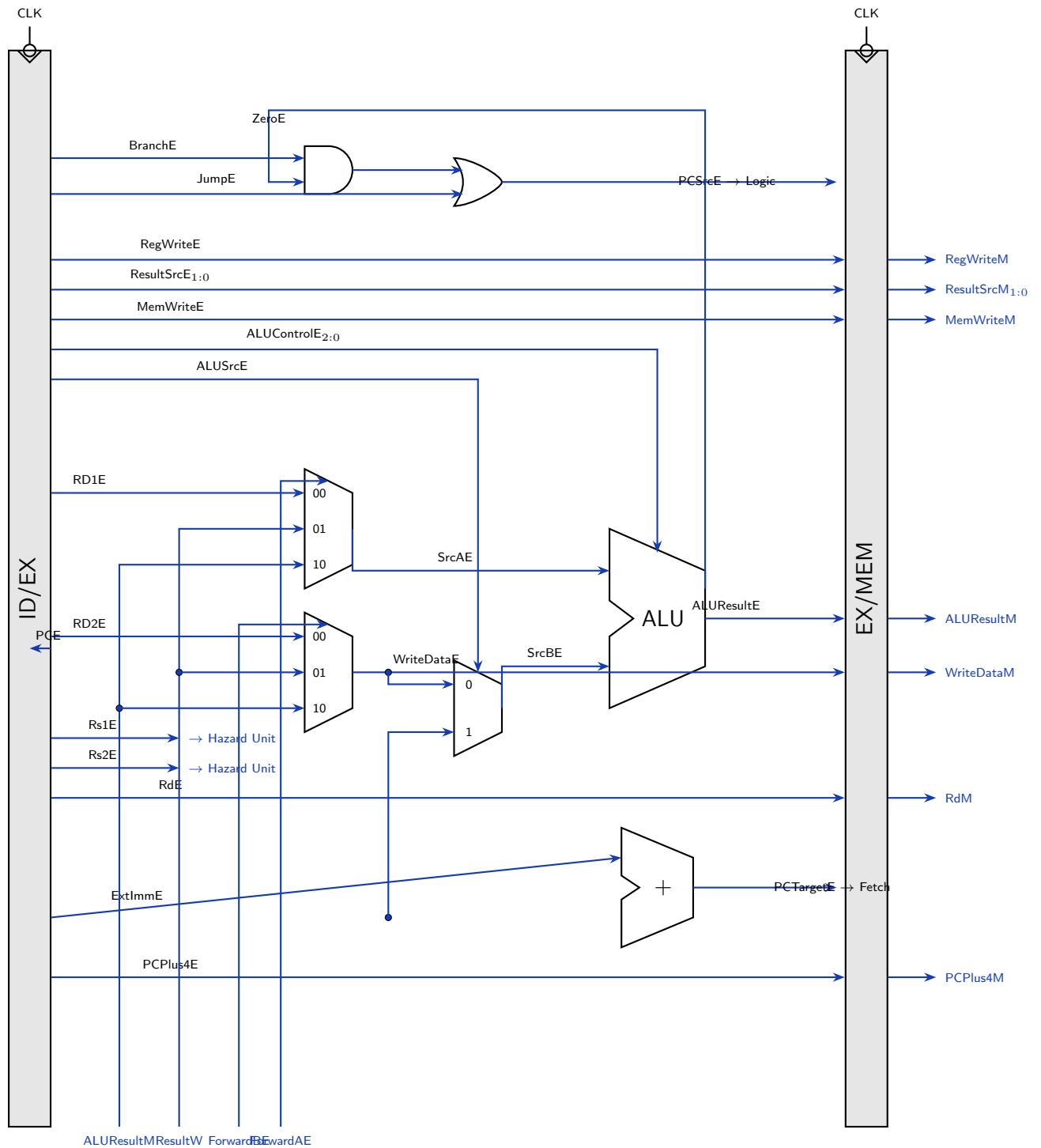


Figura 5: Etapa Execute. Muestra los multiplexores de Forwarding integrados antes de la ALU.

6.2. Pseudocódigo Completo del Execute

```

1 EXECUTE(clk, reset, FlushE, ForwardAE, ForwardBE,
2         senales_control_D, datos_D, ALUResultM, ResultW):
3
4 // 1. Registro ID/EX (flopenrc: sin stall, con flush)
5 en cada flanco de subida de clk:
6     si reset o FlushE:
7         todas las senales = 0      // Insertar burbuja NOP
8     sino:
9         capturar todas las senales de Decode
10
11 // 2. Forwarding Mux A
12 si ForwardAE == 10: SrcAE = ALUResultM // de Memory
13 sino si ForwardAE == 01: SrcAE = ResultW // de Writeback
14 sino: SrcAE = RD1E // del registro
15
16 // 3. Forwarding Mux B
17 si ForwardBE == 10: WriteDataE = ALUResultM
18 sino si ForwardBE == 01: WriteDataE = ResultW
19 sino: WriteDataE = RD2E
20
21 // 4. Selecccion del operando B de la ALU
22 si ALUSrcE == 1: SrcBE = ExtImmE // Inmediato
23 sino: SrcBE = WriteDataE // Registro
24
25 // 5. Ejecutar ALU
26 ALUResultE = ALU(SrcAE, SrcBE, ALUControlE)
27 ZeroE = (ALUResultE == 0)
28
29 // 6. Calcular direccion de salto
30 PCTargetE = PCE + ExtImmE
31
32 // 7. Decidir si se toma el salto
33 PCSrcE = (BranchE AND ZeroE) OR JumpE

```

Listing 5: Pseudocódigo de la etapa Execute

7. Etapas Memory (MEM) y Writeback (WB)

Estas dos etapas finales son responsables de confirmar los resultados arquitectónicos, ya sea interactuando con la jerarquía de memoria o actualizando el banco de registros.

Teoría de Memoria y Sincronismo: La Memoria de Datos representa típicamente la caché L1 de datos en un procesador real. En nuestra implementación, la memoria es **síncrona para escritura y asíncrona para lectura**.

- **Escritura:** Solo ocurre en el flanco de subida del reloj cuando MemWriteM está en alto. Esto evita que señales inestables modifiquen el estado de la memoria.
- **Lectura:** Las memorias reales modernas suelen ser síncronas para lectura también (añadiendo latencia), pero nuestra implementación combinacional simplificada asume que el dato se lee instantáneamente si la dirección (ALUResultM) es válida.

7.1. Componentes Activos

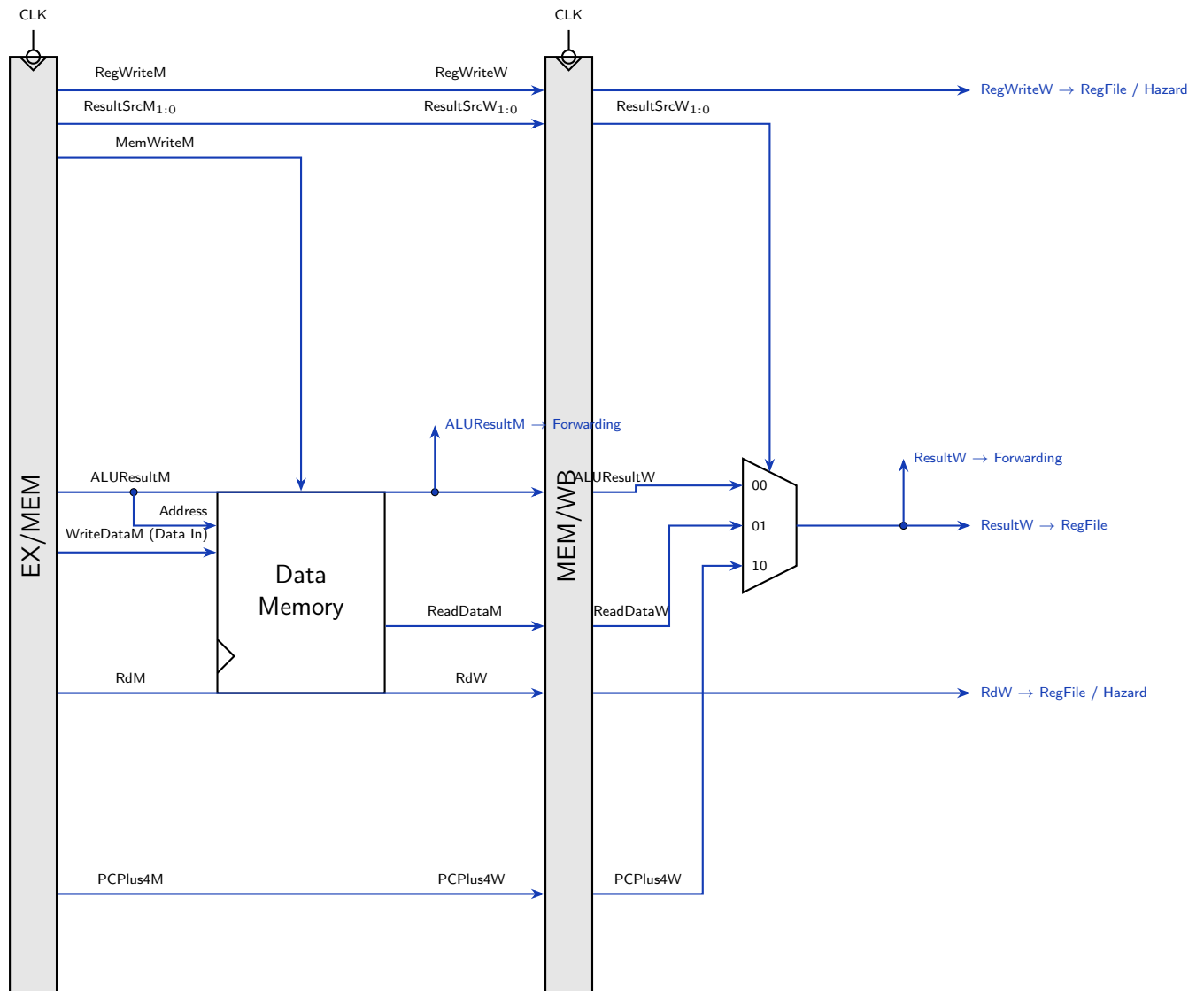


Figura 6: Etapas Memory y Writeback. La memoria de datos tiene escritura síncrona.

7.2. Pseudocódigo de ambas etapas

```

1 MEMORY(clk, reset, senales_Execute):
2   // 1. Registro EX/MEM (flop: siempre avanza)
3   en cada flanco de subida de clk:
4     si reset: todo = 0
5     sino:   capturar senales de Execute
6
7   // 2. Acceso a Memoria de Datos
8   ReadDataM = DataMemoria[ALUResultM / 4] // Lectura combinacional
9   si MemWriteM == 1:                       // Escritura síncrona
10    DataMemoria[ALUResultM / 4] = WriteDataM
11
12 WRITEBACK(clk, reset, senales_Memory):

```

```

13 // 1. Registro MEM/WB (flop: siempre avanza)
14 en cada flanco de subida de clk:
15     si reset: todo = 0
16     sino:     capturar senales de Memory
17
18 // 2. Seleccionar resultado final
19 si ResultSrcW == 00: ResultW = ALUResultW // Aritmetica
20 sino si ResultSrcW == 01: ResultW = ReadDataW // Load (lw)
21 sino si ResultSrcW == 10: ResultW = PCPlus4W // JAL

```

Listing 6: Pseudocódigo de Memory y Writeback

8. Unidad de Riesgos (Hazard Unit)

La Hazard Unit monitorea las instrucciones en curso y resuelve dependencias a través de tres mecanismos: Forwarding, Stalls y Flushes.

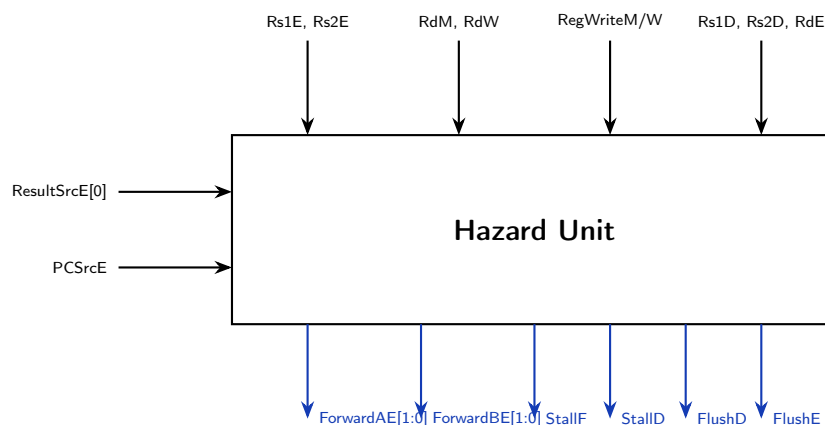


Figura 7: Diagrama de la Hazard Unit y sus múltiples conexiones.

8.1. Mecanismo 1: Forwarding (Reenvío de Datos)

El forwarding (o bypassing) es la solución de hardware más elegante para resolver los **Data Hazards** tipo RAW. Teóricamente, un resultado se calcula en la etapa Execute (ALU), pero arquitectónicamente no se escribe en el Register File hasta la etapa de Writeback (2 ciclos después). Si una instrucción posterior necesita este resultado inmediatamente, no puede leer el registro porque contiene un valor "viejo".

La Hazard Unit detecta que el registro destino de una instrucción en MEM o WB (*RdM* o *RdW*) coincide con los registros fuente de la instrucción actualmente en Execute (*Rs1E* o *Rs2E*). Al detectarlo, **cortocircuita** los multiplexores para inyectar el dato recién salido de la ALU (o de la Memoria) directamente en la entrada de la ALU de la instrucción actual, saltándose la espera por el Writeback.

```

1 // Forwarding para operando A (SrcAE)
2 si (Rs1E == RdM) Y RegWriteM Y (Rs1E != 0):
3     ForwardAE = 10 // Tomar resultado de Memory

```

```

4 sino si (Rs1E == RdW) Y RegWriteW Y (Rs1E != 0):
5     ForwardAE = 01      // Tomar resultado de Writeback
6 sino:
7     ForwardAE = 00      // Sin forwarding (usar registro)
8
9 // Forwarding para operando B (misma logica con Rs2E)

```

Listing 7: Lógica de Forwarding

8.2. Mecanismo 2: Stall (Load-Use Hazard)

El forwarding no puede resolver el Load-Use hazard (cuando un `lw` es seguido inmediatamente por una instrucción que lo usa). En ese caso, la Hazard Unit congela (stall) el PC y el IF/ID, e inserta una burbuja (flush) en el ID/EX.

```

1 // ResultSrcE[0] == 1 indica que la instruccion es un lw
2 lwStall = ResultSrcE[0] Y ((Rs1D == RdE) 0 (Rs2D == RdE))
3
4 StallF = lwStall      // Congelar PC (no avanza)
5 StallD = lwStall      // Congelar IF/ID (mantener instruccion)
6 FlushE = lwStall      // Vaciar ID/EX (insertar burbuja NOP)

```

Listing 8: Lógica de detección de Load-Use Hazard

8.3. Mecanismo 3: Flush (Control Hazard por Saltos)

Cuando se toma un salto (`PCSrcE=1`), las instrucciones que entraron especulativamente al Fetch y Decode deben descartarse, vaciando sus registros.

```

1 FlushD = PCSrcE      // Vaciar IF/ID (descarta especulativa)
2 FlushE = lwStall 0 PCSrcE // Vaciar ID/EX

```

Listing 9: Lógica de flush por salto tomado

8.4. Resumen de Mecanismos

Problema	Solución	Señales	Penalización
Data Hazard (R-type)	Forwarding	ForwardAE, ForwardBE	0 ciclos
Load-Use Hazard	Stall + Forwarding	StallF, StallD, FlushE	1 ciclo
Control Hazard	Flush	FlushD, FlushE	2 ciclos

9. Verificación: Testbench

El procesador se verificó con un testbench que ejecuta `riscvtest.txt`. El criterio de éxito es alcanzar el ciclo donde se escribe el valor 25 en la dirección 100 de la memoria. Resultado final de simulación:

*** SIMULACION EXITOSA: mem[100] = 25 ***

10. Jerarquía de Archivos

```
Microarchitecture-Pipelined/
|-- Datapath/
|   |-- riscv_pipeline.v      (Top module)
|   |-- tb.v                  (Testbench)
|   |-- flopr.v, flopenr.v, flopenrc.v (Flip-flops)
|   |-- mux2.v, mux3.v, adder.v, alu.v, extend.v
|   |-- Fetch/    fetch.v, imem.v
|   |-- Decode/   decode.v, regfile.v
|   |-- Execute/  execute.v
|   |-- Memory/   memory.v, dmem.v
|   |-- WriteBack/ writeback.v
|-- Controller/   controller.v
|-- Hazard Unit/  hunit.v
|-- Single Cycle/ riscvtest.txt
```

11. Conclusiones

1. **Pipeline vs. Single-Cycle:** La arquitectura pipelined incrementa significativamente el rendimiento dividiendo la ejecución en 5 etapas solapadas, al coste de añadir complejidad con la Hazard Unit.
2. **Manejo de Riesgos:** El forwarding es eficiente porque evita ciclos perdidos, pero el procesador aún debe detenerse 1 ciclo ante un Load-Use hazard y perder 2 ciclos ante saltos tomados.
3. **Modularidad:** El diseño permite abstraer completamente los detalles de forwarding dentro de la etapa Execute mediante los multiplexores dedicados (MUX FwdA/B).